

INDISTINGUISHABILITY OBFUSCATION

ALEX HU AND CLAIRE ZHANG

ABSTRACT. We study program obfuscation, a “holy-grail” scheme in cryptography that enables simple constructions of many cryptographic applications. We start by showing that black-box obfuscation is not possible. Then, we define indistinguishability obfuscation as the next best with. We give an overview the history of iO research, and introduce recent approach to achieving iO using functional encryption. Lastly, we explain technical challenges of the initial approach and how recent constructions address them, focusing on [10] which uses the fewest assumptions to date - Learning Parity with Noise, the existence of a Pseudo-Random Generator (PRG) with polynomial stretch, and the Decision Linear (DLIN) assumption on symmetric bilinear groups.

1. INTRODUCTION

Program obfuscation is the idea of transforming some program into a program with equivalent behavior, such that the transformed program reveals nothing about the original beyond the corresponding output for each input. Program obfuscation can be applied to all of cryptography such as holomorphic encryption, one-way functions, public key encryption, software watermarking, and more. For example, a developer could insert a watermark into a program, obfuscate it, and then distribute it, so that users cannot remove the watermark from the program without destroying functionality.

However, the aforementioned way of defining it cannot be done for all programs, so a weaker version called Indistinguishability Obfuscation (iO) was invented that could still solve a lot of cryptography problems [3]. Indistinguishability Obfuscation says that any two programs with equivalent behavior must be obfuscated so that they are computationally indistinguishable. Also, iO has also been useful in theoretical questions, such as by being the first cryptographic evidence of the average-case hardness of the complexity class PPAD that contains the problem of finding Nash equilibria by constructing hard instances for the end of the line problem [5].

The first time iO was constructed was in 2013 [4]. Some constructions were based on assumptions related to multilinear maps, but the trend is towards using more standard cryptographic assumptions now, and security issues have been found with some multilinear maps. A common thread among many more recent iO constructions is using a reduction to any public-key functional encryption scheme that is “compact,” which means the runtime of encryption is polynomial in the security parameter and the input size and doesn’t need to depend on the circuit size or output length of the function [2]. Here is a timeline:

- (1) **2001 — VBB Impossibility.** Barak *et al.* prove that virtual black-box obfuscation for general circuits is impossible, thereby motivating weaker notions such as iO [3].
- (2) **2013 — First Candidate iO.** Garg *et al.* give the first candidate construction of iO and functional encryption for all circuits based on graded encoding schemes [4].
- (3) **2015 — iO from Multilinear Maps.** Gentry, Bishop Lewko, Sahai, and Waters realise iO under the *multilinear subgroup elimination* assumption, providing the first concrete security reduction [7].
- (4) **2017 — iO from Trilinear Maps.** Lin and Tessaro lower the map degree to 3 and introduce block-wise local PRGs to streamline the construction [11].
- (5) **2018 — iO *without* k -Linear Maps.** Ananth, Jain, and Sahai eliminate higher-degree maps altogether, achieving iO from sub-exponential LWE, bilinear maps, and a perturbation-resilient generator [1].

- (6) **2020 — iO from Well-Founded Assumptions.** Jain, Lin, and Sahai obtain iO from four well-studied assumptions—sub-exp. LWE, sub-exp. LPN, SXDH, and an NC^0 PRG—moving the field beyond heuristic graded encodings [8].
- (7) **2021 — Indistinguishability Obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0** The same authors go further and remove LWE, constructing iO (and even FHE) from sub-exp. LPN over $\mathbb{F}_p$, the DLIN assumption on symmetric pairings, and an NC^0 PRG [10].

2. BLACK-BOX OBFUSCATION

The strongest/“ideal” version of program obfuscation is blackbox obfuscation.

Definition 2.1 (Negligible function). A function $\alpha : \mathbb{N} \rightarrow \mathbb{R}^+$ is negligible if $\forall c > 0, \exists n_0, \forall n \geq n_0, \alpha(n) \leq n^{-c}$.

Definition 2.2 (Blackbox Obfuscator for Turing Machines). A probabilistic algorithm $\mathcal{O}$ is an obfuscator [3] if

- For every TM M , $\mathcal{O}(M)$ is a TM that computes the same function as M .
- For every TM M , $|\mathcal{O}(M)| \leq \text{poly}(|M|)$, and if M halts in t steps on x , then $\mathcal{O}(M)$ halts in at most $\text{poly}(t)$ steps on x .
- For any probabilistic polynomial time (PPT) algorithm A , there is a PPT S and a negligible function α such that for all TMs M , where S^M is S with blackbox access to M ,

$$\left| \Pr[A(\mathcal{O}(M)) = 1] - \Pr[S^M(1^{|M|}) = 1] \right| \leq \alpha(|M|).$$

We say $\mathcal{O}$ is efficient if it runs in polynomial time.

Definition 2.3 (Blackbox Obfuscator for Circuits). A probabilistic algorithm $\mathcal{O}$ is an obfuscator [3] if

- For every circuit C , $\mathcal{O}(C)$ is a circuit that computes the same function as C .
- For every circuit C , $|\mathcal{O}(C)| \leq \text{poly}(|C|)$.
- For any probabilistic polynomial time (PPT) algorithm A , there is a PPT S and a negligible function α such that for all circuits C ,

$$\left| \Pr[A(\mathcal{O}(C)) = 1] - \Pr[S^C(1^{|C|}) = 1] \right| \leq \alpha(|C|).$$

We say $\mathcal{O}$ is efficient if it runs in polynomial time.

Theorem 2.4 (Existence of unobfuscatable programs). *For every $\mathcal{O}$, there exists P such that $\mathcal{O}$ fails to obfuscate P .*

Proof. Define a family $\{P_{x,y}\}$ where x and y are n -bits as

$$P_{x,y}(b, \Pi) = \begin{cases} y & \text{if } b = 0 \text{ and } \Pi = x \\ x, y & \text{if } b = 1 \text{ and } \Pi(0, x) = y \\ 0 & \text{otherwise.} \end{cases}$$

For any $\mathcal{O}$, if we let $P' = \mathcal{O}(P_{x,y})$, then there's some x, y such that S having black-box access to $P_{x,y}$ will not know y with probability at least $1 - \alpha(|P|)$ for some negligible α , since S has polynomial runtime and there's 2^n possible x 's and y 's. However, if we're given P' , we can run $P'(1, P') = (x, y)$. $\square$

3. INDISTINGUISHABILITY OBFUSCATION

Indistinguishability obfuscation (iO) can be thought of as not hiding the entire truth table all at once, but hiding the particular implementation.

Definition 3.1 (Computational, Subexponential, ϵ -indistinguishability). We say two ensembles $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ are ϵ -indistinguishable if for every PPT P , for sufficiently large λ ,

$$\left| \Pr_{x \in \mathcal{X}_\lambda} [A(1^\lambda, x) = 1] - \Pr_{y \in \mathcal{Y}_\lambda} [A(1^\lambda, y) = 1] \right| \leq \epsilon(\lambda).$$

They're computationally indistinguishable if ϵ is negligible, and subexponentially indistinguishable if $\epsilon = 2^{-\lambda^c}$ for some $c > 0$. We will use $\approx_c$ to denote this.

Definition 3.2 (Indistinguishability Obfuscator for Circuits). A *uniform PPT* algorithm iO is called a (T, γ) -secure indistinguishability obfuscator for polynomial-sized circuits if the following holds:

- **Completeness.** For every $\lambda \in \mathbb{N}$, every circuit C with input length n , and every input $x \in \{0, 1\}^n$, we have

$$\Pr[C'(x) = C(x) : C' \leftarrow iO(1^\lambda, C)] = 1.$$

- **(T, γ) -Indistinguishability.** For every two ensembles $\{C_{0,\lambda}\}, \{C_{1,\lambda}\}$ of polynomial-sized circuits that have the same size, input length, and output length, and are functionally equivalent (i.e., $C_{0,\lambda}(x) = C_{1,\lambda}(x)$ for all inputs x), the following two distributions are (T, γ) -indistinguishable:

$$\{iO(1^\lambda, C_{0,\lambda})\} \quad \text{and} \quad \{iO(1^\lambda, C_{1,\lambda})\}.$$

An example of the power of iO is from [12]:

Theorem 3.3. *If iO and a length-doubling PRG exist, so does public-key encryption.*

Proof. Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ be a length-doubling PRG. Let the secret key be $sk = s$, sampled from $\{0, 1\}^n$, and the public key be $pk = G(s)$. Let $P(x)$ be the program that outputs m if $G(x) = pk$ and otherwise 0. Let the encoding function output $P' = iO(P)$. The decoding function then runs $P'(sk) = m$.

For security purposes, we want to show that $\{(pk, Enc_{pk}(m_0))\}_n$ and $\{(pk, Enc_{pk}(m_1))\}_n$ are computationally indistinguishable. By PRG security, $\{(pk, Enc_{pk}(m_0))\}_n$ and $\{(y, Enc_y(m_0))\}_n$ are computationally indistinguishable where y is uniformly random. Now, consider $\{(y, Enc_y(m_0))\}_n$ and $\{(y, Enc_y(m_1))\}_n$; with probability $1 - 1/2^n$ both output 0. $\square$

If we only want an inefficient indistinguishability obfuscator, we can construct it quickly:

Theorem 3.4. *If $P = NP$, iO exists.*

Proof. For any circuit C let $iO(C)$ be the lexicographically first circuit C' that computes the same function as C . $\square$

4. iO CONSTRUCTION

We follow the work [10] since it's recent and relies on relatively weaker assumptions. A lot of prior works such as [8] rely on an assumption of Learning With Errors (LWE), a well-known assumption in cryptography. Figure 1 is a broad overview of their construction and when different parts were discovered.

We will sample a lot from uniform distributions from some set, so we will use $\leftarrow$ to denote this. The three assumptions that [10] uses are:

Definition 4.1 (Existence of Boolean PRG in NC^0 with polynomial stretch). A polynomial-stretch PRG in NC^0 is a function $\{0, 1\}^n \rightarrow \{0, 1\}^{m(n)}$ where $m(n) = n^{1+\Omega(1)}$ and each output bit depends on a constant number of input bits such that $\{PRG(r)\}_{r \leftarrow \{0, 1\}^n} \approx_c \{r'\}_{r' \leftarrow \{0, 1\}^{m(n)}}$.

Definition 4.2 (Learning Parity with Noise (δ -LPN) assumption). Let $\delta \in (0, 1)$. For any constants $\eta_p, \eta_n > 0$, any function $p : \mathbb{N} \rightarrow \mathbb{N}$ where $\forall \ell \in \mathbb{N}, p(\ell)$ is a prime of ℓ^{η_p} bits, letting $p = p(\ell), n = \ell^{\eta_n}, r = \ell^{-\delta}$, we have:

$\{(A, b = s \cdot A + e) | A \leftarrow \mathbb{Z}_p^{\ell \times n}, s \leftarrow \mathbb{Z}_p^{1 \times \ell}, e \leftarrow \mathcal{D}_r^{1 \times n}(p)\}_{\ell \in \mathbb{N}} \approx_c \{(A, u) | A \leftarrow \mathbb{Z}_p^{\ell \times n}, u \leftarrow \mathbb{Z}_p^{1 \times n}\}_{\ell \in \mathbb{N}}$ and if they're subexponentially indistinguishable subexponential δ -LPN holds.

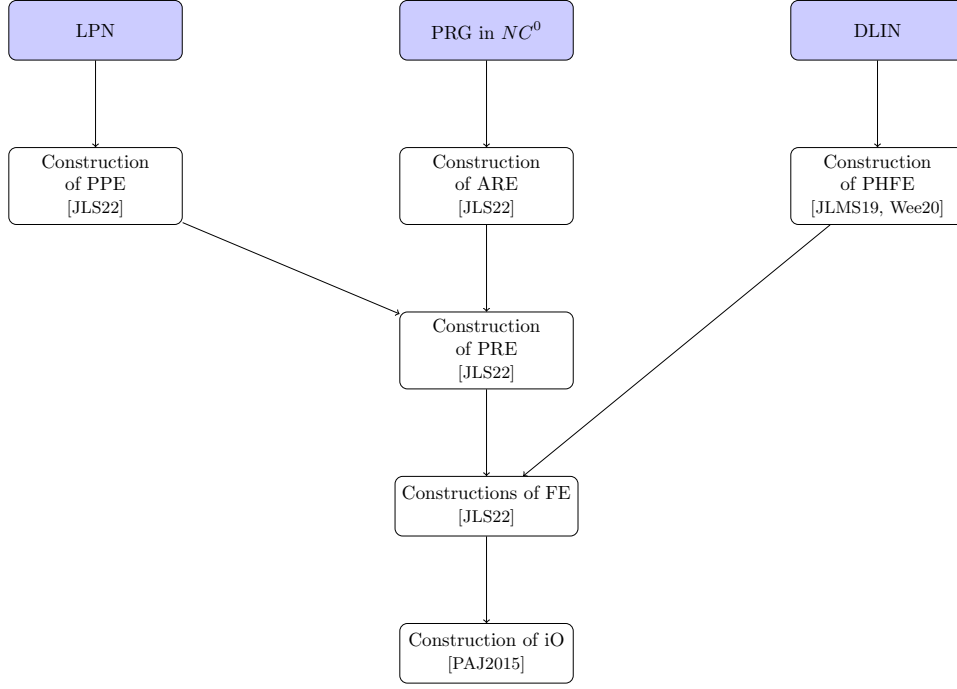


FIGURE 1. Diagram of [10] iO Construction

This assumption takes many forms. This is a weaker version that deals with bilinear maps when the two groups are the same; there are also versions where the two groups are different.

Definition 4.3 (Decisional Linear Assumption (DLIN) [9]). Let p be a certain λ -bit prime. Let G and G_T be groups of order p such that there exists an efficiently computable nontrivial symmetric bilinear map $e : G \times G \rightarrow G_T$ with canonical generator g , i.e. $e(g^a, g^b) = e(g, g)^{ab}$ and $e(g, g) \neq 1$. Then

$$\{(g^x, g^y, g^{xr}, g^{ys}, g^{r+s} | x, y, r, s \leftarrow \mathbb{Z}_p)\} \approx_c \{g^x, g^y, g^{xr}, g^{ys}, g^z | x, y, r, s, z \leftarrow \mathbb{Z}_p\}.$$

Many recent results rely on a sublinear functional encryption (FE) for polynomial size circuits, which means encryption can be done in sublinear time in the number of inputs plus circuit size. We define the public-key case, and using secret-key encryption it is also possible to show iO [10].

Definition 4.4 (Functional encryption scheme (FE)). Let λ be the security parameter and $\mathcal{F}_{n,m}$ be a function class on $n = n(\lambda)$ input bits and $m = m(\lambda)$ output bits where n and m are polynomials, and where every output bit can be computed by a circuit of size λ . A function encryption scheme consists of:

- $\text{Setup}(\lambda, n, m)$ outputs a public key and master secret key (PK, MSK) .
- $\text{Enc}(PK, x)$ outputs the ciphertext CT .
- $\text{KeyGen}(MSK, f)$ outputs a functional decryption key SK_f .
- $\text{Dec}(SK_f, CT)$ deterministically outputs a value $y \in \{0, 1\}^n$ or $\perp$ if it fails.

This scheme is sublinear if the size of the circuit computing $\text{Enc}(PK, \cdot)$ is $O((n + m^{1-\epsilon})\text{poly}(\lambda))$ and correct if it returns the right function output for all inputs and functions.

The definition of security resembles the definition of iO. This security ensures that adversaries can get no additional information about x from the ciphertext and secret key besides $f(x)$.

Definition 4.5 (IND security). Let A be a PPT adversary. Then A chooses two $x_0 \neq x_1$ and one function f such that $f(x_0) = f(x_1)$. Then an FE scheme is initialized. Then two ciphertexts CT_0 and CT_1 for x_0 and x_1 and a secret key for f SK_f is generated. Then A gets PK , a random CT_b , and SK_f and needs to decide what b is. The probability of success is at most $1/2 + \alpha(\lambda)$ where α is negligible. If $\alpha = 2^{-\lambda^{\Omega(1)}}$ then the FE is subexponentially secure.

Theorem 4.6 ([2]). *If there exists a subexponentially-secure sublinear functional encryption scheme then there exists an indistinguishability obfuscation scheme.*

4.1. Working towards Sublinear FE. Many iO constructions construct weaker versions of FE for simpler functions and then bootstrap to all functions. There is a generalization of FE called Partially Hiding Functional Encryption (PHFE) where the input consists of an additional public input PI that does not need to be hidden and also a secret input SI and encryption time is proportional to $|PI| + |SI|$. The work [10] uses a prior PHFE construction such as [5] for functions that are polynomials over prime fields that have constant degree in PI and degree two in SI .

Syntax of a PHFE Scheme

A public-key *partially hiding functional encryption* scheme (**PHFE**) for the functionality $\mathcal{F}_{\text{PHFE}}$ consists of the following polynomial-time algorithms:

- **PPGen**(1^λ) (*public-parameter generation*)
A randomized algorithm that takes the security parameter λ and outputs $PP = (\text{crs}, p)$, where p is a modulus.
- **Setup**($d, 1^{n_{\text{PHFE}}}, PP$) (*system setup*)
A randomized algorithm that takes a degree $d \in \mathbb{N}$, a length parameter n_{PHFE} , and $PP = (\text{crs}, p)$. These parameters define the function class $\mathcal{F}_{\text{PHFE}, d, p, n_{\text{PHFE}}}$. The algorithm outputs a public key PK and a master secret key MSK .
- **Enc**($PK, (PI, SI) \in \mathbb{Z}_p^{n_{\text{PHFE}}} \times \mathbb{Z}_p^{n_{\text{PHFE}}}$) (*encryption*)
A randomized algorithm that takes the public key PK and a message (PI, SI) , producing a ciphertext CT . The value PI is treated as *public* input, while SI is *secret*. The ciphertext is assumed to reveal PI in the clear.
- **KeyGen**($MSK, f \in \mathcal{F}_{\text{PHFE}, d, p, n_{\text{PHFE}}}$) (*function-key generation*)
A randomized algorithm that, on input a degree- $(d, 2)$ polynomial f over $\mathbb{Z}_p$, returns SK_f , a secret key that enables decryption of f .
- **Dec**(SK_f, CT) (*decryption*)
A deterministic algorithm that outputs a value $\text{out} \in \mathbb{Z}_p \cup \{\perp\}$.

Notably, we want a stronger FE that doesn't require f to be degree- $(d, 2)$ (and supports full privacy of x). If we can somehow encode an arbitrary f in a format from which it can be evaluated with a degree- $(d, 2)$ polynomial, and convert $x \rightarrow PI, SI$, we can directly use PHFE in our (full) FE construction.

4.1.1. Processed Randomized Encodings. A Randomized Encoding (RE) is of a function f computed by circuit C evaluated at x is a transformation $\pi(x, C)$, from which $C(x)$ can be computed. As of now, known RE schemes are at least degree 4 in SI . In 2021, [10] makes a key contribution of constructing Preprocessed Randomized Encoding (PRE) which generates PI, SI from x and p (some prime) where $C(x)$ can be computed from PI, SI with constant and degree two, exactly the degree requirements of PHFE. The PRE is then fed into the PHFE to produce FE (PRE.Encode is precisely f in PHFE). The PRE preprocess must run in $o(m)$ and encode h in $O(m)$ so that the FE is sublinear; we can be more lax about the encode part because PHFE guarantees a sublinear evaluation of h . To maintain the IND security of FE, since PHFE is secure, PRE must satisfy:

Definition 4.7 (IND security of PRE). For any circuit C and inputs x_0, x_1 with $C(x_0) = C(x_1)$, the distributions of (PI, π) produced from (C, x_0) and (C, x_1) are computationally indistinguishable.

Preprocessed Randomized Encoding

- **PRE.PreProc**(p, x) $\rightarrow$ (PI, SI). Given an input $x \in \{0, 1\}^n$ and randomness r , the algorithm outputs the preprocessed pair (PI, SI) $\in \mathbb{Z}_p^2$. Preprocessing *does not* depend on the circuit to be encoded (only on an upper bound for its size). It satisfies *sublinear time-succinctness*:

$$T_{\text{PreProc}} = m^{1-\varepsilon} \text{poly}(\lambda).$$

- **PRE.Encode**(C, PI, SI) = π . On input a circuit C of size $m \cdot \text{poly}(\lambda)$ and the preprocessed pair (PI, SI), the encoder outputs a binary string π . Computation is a polynomial map over $\mathbb{Z}_p$ of constant degree in PI and degree 2 in SI.
- **PRE.Decode**(π) = y . The decoder recovers $y = C(x)$ from π .

Our desired FE scheme relying on a PRE can be summarized in the following diagram.

PRE-based FE Scheme

- FE.Setup**($1^\lambda, 1^{n_{\text{FE}}}, 1^{m_{\text{FE}}}$): • PHFE.PPGen(1^λ) $\rightarrow$ PHFE.PP = (crs, p),
- PHFE.Setup($d, 1^{n^{\text{PHFE}}}, \text{PP}$) $\rightarrow$ (PHFE.PK, PHFE.MSK),
 - Output PK = (PHFE.PK, crs, p) and MSK = PHFE.MSK.
- FE.Enc**(PK, $x \in \{0, 1\}^{n_{\text{FE}}}$): • Parse PK = (PHFE.PK, crs, p),
- PRE.PreProc($1^\lambda, 1^{n_{\text{PRE}}}, 1^{m_{\text{PRE}}}, 1^{k_{\text{PRE}}}, p, x$) $\rightarrow$ (PI, SI),
 - PHFE.Enc(PHFE.PK, (PI, SI)) $\rightarrow$ CT,
 - Return CT.
- FE.KeyGen**(MSK, C): • Let $f_1, \dots, f_T$ be the degree- $(d, 2)$ polynomials that compute PRE.Encode($C, (\cdot, \cdot)$);
- For each $i \in [T]$ compute PHFE.KeyGen(PHFE.MSK, f_i) $\rightarrow$ SK $_i$;
 - Output SK $_C$ = (SK $_1, \dots$, SK $_T$).
- FE.Dec**(SK $_C$, CT): • Parse SK $_C$ = (SK $_1, \dots$, SK $_T$),
- For every $i \in [T]$ set y_i = PHFE.Dec(SK $_i$, CT),
 - Let $y = (y_1, \dots, y_T)$ and output PRE.Decode(y).

Now, we describe the ideas building up to the final PRE scheme used in PHFE [10]. We start with the initial idea from which [8] and [10] were both built on top of.

4.2. PRE: the first pass. The paper [8] turns to Yao's Garbled Circuit – foundational cryptographic tool that has existed for decades – as the starting point for the RE.

4.2.1. Yao's Garbled Circuit. Yao's garbled circuit (Yao's GC) is most commonly constructed as a multi-party computation protocol, where n parties, the i th holding some binary number $\mathbf{x}_i$, jointly compute $f(\mathbf{x}_1, \dots, \mathbf{x}_n)$. No one should know anything else besides (what can be simulated and) their $\mathbf{x}_i$ and $f(\mathbf{x}_1, \dots, \mathbf{x}_n)$. For its purpose as a RE, there is essentially only one person, so we omit description of oblivious transfer (OT), which is an essential part of the multi-party aspect of Yao's GC.

We define a function **Garble** that can turn a circuit and input to some string from which the evaluation can be efficiently computed, while hiding the input, i.e. $\text{Garble}(C, x) = \pi$, where $f(\pi) = C(x)$ for a polynomial (in $O(|C| + |x|)$) time algorithm f . Consider a circuit which is a single AND gate, with inputs x_L and x_R (bits).

x_L	x_R	$x_L \wedge x_R$	Garbled output	Permuted garbled output
0	0	0	$\text{Enc}(H(W_L^0, W_R^0), 0)$	$\text{Enc}(H(W_L^0, W_R^0), 0)$
0	1	0	$\text{Enc}(H(W_L^0, W_R^1), 0)$	$\text{Enc}(H(W_L^0, W_R^1), 1)$
1	0	0	$\text{Enc}(H(W_L^1, W_R^0), 0)$	$\text{Enc}(H(W_L^1, W_R^0), 0)$
1	1	1	$\text{Enc}(H(W_L^1, W_R^1), 1)$	$\text{Enc}(H(W_L^1, W_R^0), 0)$

In the above diagram, $H(x)$ is any hash function and $\text{Enc}(\text{sk}, \text{msg})$ is any encryption function.

If we associate a random string W to each input-bit pair, then knowing $W_L^{x_L}$ and $W_R^{x_R}$ lets you decrypt and know $x_L \wedge x_R$: try to decrypt each of the 4 encryptions; one of them returns a bit, the others are nonsense. Thus, a valid encoding of $(C = \wedge, x_L, x_R)$ is $\pi = W_L^{x_L} || W_R^{x_R} || \text{Permuted garbled output}$. Notably, viewing $\mathbf{x} = (x_L, x_R)$, $\mathbf{s} = W$, each output bit depends on at most two x_i s and $|W|$ random bits. This is crucial to note for our PRE Encode locality constraint.

We can extend this idea to a chain of gates: instead of the garbled output being an encryption of the actual output of the gate, we let it be an encryption of the next $W_{L/R}^{0/1}$ 0/1 chosen based on x , to be used as the random strings in upstream gate garbles. Sweeping the security proof aside, Yao's GC gives us a $(O(1), O(1))$ -degree RE.

4.2.2. Yao's GC as PRE. We now analyze its potential of being our desired PRE. If any of $x' := x, r$ are public, an adversary can learn more than just $C(x)$ from π and x . However, Garble is degree-4 in x' , so while we have constant locality in all x, r , Yao's GC is degree-4 in inputs we need to keep private. We need to do more work.

4.3. Hiding inputs: the second pass. The solution to the hiding vs. degree-2 tradeoff of Yao's GC will be to let x' be the public (pre-processed) input PI. But in a nuanced way.

4.3.1. Fully Homomorphic Encryption. Fully Homomorphic Encryption (FHE) is another cryptographic primitive that has been used since 2009 [6]. It enables arbitrary computations on encrypted data without requiring decryption. Unlike traditional encryption schemes, which only support encryption and decryption, FHE allows a third party to evaluate functions over ciphertexts while preserving input-hiding. Formally, given ciphertexts $\text{Enc}(m_1)$ and $\text{Enc}(m_2)$ and arbitrary function f ,

$$\text{Enc}(f(m_1, m_2)) = f'(\text{Enc}(m_1), \text{Enc}(m_2)).$$

Assumption 4.2 is commonly used to construct FHE:

$$\text{PI} = \text{HEEnc}(x') = (A, b := s \cdot A + e + x \mod p).$$

There are well-known methods that enable computing encryptions of arbitrary functions of x . If FHE is able to do arithmetic field operations on the encrypted PI, and these operations could be of high degree, much larger than two, we bypass the degree restriction of 2. Then, our SI can store the information needed to decrypt the FHE scheme such that decryption can be done in constant degree in PI and degree 2 in SI. We can express Yao's Garbled Circuit encryption as the linear combination of constant degree polynomials of x and sublinear random bits r .

Formally, consider a revised PRE protocol, combining Yao's GC as a base RE, denoted Encode' , and FHE:

- Circuit C , input x .
- $\mathbf{x}' := (x, r)$ where r are the random bits used in Yao's garbling of C .
- $\text{PRE.PreProc}(p, x)$: $\text{PI} = \text{HEEnc}(x')$, $\text{SI} = s$.
- $\text{PRE.Encode}(C, \text{PI}, \text{SI})$: Homomorphically evaluate $\text{HEEnc}(\pi)$, where $\pi := \text{Encode}'(C)$, then (FHE) decrypt to obtain π .

Note we need to ensure:

- PI, SI can be computed by a circuit of size sublinear in size of C , and
- Homomorphic evaluation of a locality $d = O(1)$ function Encode' is a degree d operation on PI , and decryption is a degree 2 operation in SI .

From here, we can treat Encode' as an arbitrary degree $d = O(1)$ polynomial $h(x')$. In PRE.Encode , we (homomorphically) compute $h(x')$. This can be done under our sublinear time constraint by squeezing all degree- d monomials in SI . By choosing the FHE dimension $(A \in \mathbb{F}_p^{\dim \times |x'|})$ to be polynomially smaller than x' , the above SI be sufficiently succinct.

However, FHE decryption turns out to not be sufficiently sound. To fix this, [8] propose $SI = (1||s)^{\oplus d}, U, V$, where U, V contain information to more accurately decode $h(x') \rightarrow x'$. [8] ensures U, V are sublinear in size but not time – evaluating U, V takes $\Omega(m)$ time.

4.4. Sublinear preprocess time: the third pass. The issue of the previous protocol is that the FHE decryption doesn't have enough information to accurately decode, so we can instead want the preprocessed inputs to provide the needed information:

$$\text{Corr} = \text{Encode}_C^0(x^0 + e) - \text{Encode}_C^0(x^0)$$

However, this introduces two issues.

- (1) The runtime of PRE.PreProc is $\Omega(|C|) = \Omega(m \cdot \text{poly}(\lambda))$.
- (2) PRE.PreProc depends on C .

[8] resolves (1) by transforming size-succinct FHE to time-succinct FHE, assuming LWE. [10] solves both issues directly, without requiring additional assumptions, by preprocessing *a batch of inputs at a time*.

The goal is now to construct a Preprocessed Polynomial Encoding (PPE):

Preprocessed Polynomial Encoding

- **PPE.PreProc** $(p, Q, x_1, \dots, x_k) \rightarrow (\text{PI}, \text{SI})$. Given a collection of constant-degree- d monomials $Q = \{Q_1, \dots, Q_{m'}\}$, the preprocessing algorithm converts a batch of k inputs $\{x_i \in \{0, 1\}^n\}_{i \in [k]}$ into a preprocessed input (PI, SI) over $\mathbb{Z}_p$. It satisfies *sublinear time-succinctness*: preprocessing time is sublinear in $m' \cdot k$.
- **PPE.Decode** $(p, Q, h, \text{PI}, \text{SI}) = y$. The decoding algorithm recovers

$$y = h(x_1, \dots, x_k) = \sum_{i=1}^k \sum_{j=1}^{m'} \eta_{i,j} Q_j(x_i) \mod p.$$

- **Indistinguishability security**: For any two batches $(x_1, \dots, x_k)$ and $(x'_1, \dots, x'_k)$, the distributions of PI produced by PPE.PreProc are computationally indistinguishable.

The paper [10] constructs PPE by modifying h from being Yao's GC RE to an *amortized* RE (ARE). This involves multiple GCs computes on separate batches of inputs.

After these many patches, [10] achieve an iO scheme relying on minimal assumptions.

REFERENCES

- [1] Prabhanjan Ananth, Aayush Jain, and Amit Sahai. Indistinguishability obfuscation without multilinear maps: iO from LWE, bilinear maps, and weak pseudorandomness. Cryptology ePrint Archive, Report 2018/615, 2018.
- [2] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In *Advances in Cryptology-CRYPTO*, pages 308–326. Springer, 2015.
- [3] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2001.
- [4] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*. IEEE, 2013.
- [5] Romain Gay, Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from simple-to-state hard problems: New assumptions, new techniques, and simplification. In *Advances in Cryptology – EUROCRYPT 2021*, pages 97–126. Springer, 2021.
- [6] Craig Gentry. Fully homomorphic encryption using ideal lattices. *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009.
- [7] Craig Gentry, Allison Bishop Lewko, Amit Sahai, and Brent Waters. Indistinguishability obfuscation from the multilinear subgroup elimination assumption. In *56th IEEE Symposium on Foundations of Computer Science (FOCS 2015)*, pages 151–170, 2015.

- [8] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 60–73. ACM, 2020.
- [9] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. ACM, 2021.
- [10] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability Obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In *Advances in Cryptology – EUROCRYPT*, pages 670–699. Springer, 2022.
- [11] Huijia Lin and Stefano Tessaro. Indistinguishability obfuscation from trilinear maps and block-wise local PRGs. Cryptology ePrint Archive, Report 2017/250, 2017.
- [12] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In *STOC '14*, pages 475–484. ACM, 2014.

Email address: alexhu@mit.edu

Email address: clairefz@mit.edu